MedQA: Question-answering system based on medical literature

COMP6713 2026 T1 — Group Project Report

Yuyi Zhu z5629988
School of Computer Science and Engineering
UNSW Sydney
Sydney, Australia

Songning Liu z5668661
School of Computer Science and Engineering
UNSW Sydney
Sydney, Australia

Ian Zhao z5666368 *
School of Computer Science and Engineering
UNSW Sydney
Sydney, Australia

* Corresponding author

Wanxin Ma z5695666
School of Computer Science and Engineering
UNSW Sydney
Sydney, Australia

Muyi Yang z5656930
School of Computer Science and Engineering
UNSW Sydney
Sydney, Australia

Scope Declaration

The table below summarises the project scope as defined by the COMP6713 26T1 project guidelines. Claimed credits: 130 (min. 80). See SCOPE_MAPPING.md in the repository for the scope-to-code mapping.

PART A: Problem Definition (Minimum: 10 credits)			
Feature	**Credits**	**Details / Examples**	**Our project**
NLP Problem	5 per problem	Examples: Question-answering, Sentiment Analysis, etc.	Question-answering (medical literature)
Text Source / Domain	5 per source	Examples: news articles, medical papers, social media.	Medical papers (PubMed abstracts; USMLE questions)
Part A subtotal			**10**
PART B: Dataset Selection (Minimum: 20 credits)			
Two existing datasets	10	Publicly available datasets.	PubMedQA + BioASQ + MedQA-USMLE (three datasets via HuggingFace)
Create own labelled dataset	20	Correct label selection; inter-annotator agreement.	— (not claimed)
Existing lexicon	10	Examples: WordNet, medical ontology.	UMLS (Unified Medical Language System) via scispaCy UMLS linker
Part B subtotal			**20**
PART C: Modelling (Minimum: 30 credits)			
Rule-based / statistical baseline	— (essential)	Keep simple — baseline only.	TF-IDF + cosine similarity (sklearn TfidfVectorizer)
Pre-trained / fine-tuned model	5 per model	Must compare multiple models.	— (covered by fine-tune row below)
Fine-tune a model	20 per method	Examples: fine-tuned BERT, prefix-tuned LLaMA.	Fine-tune PubMedBERT on combined QA corpus (SQuAD-style span)
Extend a method	30 per method	Loss modification, structured ontology, non-trivial prompting.	Answer-type-conditioned prompting + UMLS query expansion + BGE cross-encoder rerank
Integrate LLM with external tools	20	Use LangChain / LangSmith or similar.	LangChain RAG: BGE-M3 embeddings, ChromaDB vector store, Qwen2.5-14B NF4 generator
Part C subtotal			**70**
PART D: Evaluation (Minimum: 20 credits)			
Quantitative evaluation	10	Appropriate metrics.	EM, Token-F1, BERTScore, ROUGE-L, Yes/No accuracy, Recall@k, MRR; per-source breakdown
Qualitative evaluation	5	Mis-classified instances, common error types.	Five-class heuristic (WRONG_RETRIEVAL / HALLUCINATION / PARTIAL_ANSWER / TERMINOLOGY_MISMATCH / OTHER) in medqa/evaluation/qualitative.py
Command-line testing	5	Input argument or file.	uv run medqa --question … (medqa/cli.py)
Demo	10	Gradio or equivalent.	Gradio web UI at http://localhost:7860 (main.py)
Part D subtotal			**30**
Total claimed: 130 credits (minimum required: 80)			

I. *Problem Definition*

Medical QA task's input is a sequence of natural-language medical questions, and its output should be accurate, evidence-based medical-specific answers. Compared with a general-domain QA system, there are three main difficulties in answering medical questions professionally:

A. Numerous synonyms for terminology:

The same concept has two sets of expressions. For example, patient-language ("heart attack") and clinical terminology ("myocardial infarction", "MI", "AMI"). This means that if the system relies solely on keywords in the question for matching and searching, it will miss half of the relevant literature.

B. Answer formats differ across datasets:

PubMedQA expects yes/no/maybe, BioASQ expects noun phrases of 1 to 8 words, and MedQA-USMLE is a multiple-choice question. This necessitates that a single question-answering model must be able to adapt to multiple answer formats simultaneously.

C. High standard precision:

The practical application of question-answering systems in medical scenarios should not be based on "hallucination"; answers should be theoretically grounded rather than fabricated. This implies that every output of the question-answering system must be strictly based on authoritative literature in the relevant field, and cannot rely solely on the parameterized knowledge of the LLM.

II. Datasets and Lexicons

TABLE I. TABLE A

Dataset	***Scale***	***Task type***	***Answer format***	***HF Source***
PubMedQA (labelled)	1,000	Document-based question and answer	yes/no/maybe + Long answer	qiaojin/PubMedQA
BioASQ	≈3,000	Biomedical Q&A	Factoid	kroshan/BioASQ
MedQA-USMLE	≈12,000	multiple choice	Choosing from A/B/C/D	HuggingFace
PubMedQA (unlabelled)	≈61,000	For use only in RAG corpus	n/a	qiaojin/PubMedQA

Annotation quality and inter-annotator agreement:

This project does not rely on self-built annotation sets but rather publicly available datasets. Hence, the consistency measures that are reported by the original publishers can be summarized as follows:

A. PubMedQA (Jin et al. 2019):

PubMed paper titles are used as questions, while models are tasked to infer the decision (yes/no/maybe); this dataset contains 1,000 expert-annotated samples, all of which have successfully passed consistency tests (Cohen's Kappa of 0.78, indicating substantial agreement).

BERT Labeling Strategy: The span labels required to train PubMedBERT are automatically generated from long answers written by experts (primarily from PubMedQA and BioASQ) through deterministic matching logic in the data pipeline. While this rule-based approach guarantees 100% inter-annotator consistency by definition, it introduces potential label noise (assigned to the [CLS] tag) when a completely matching string cannot be found in the truncated context.

B. BioASQ (Tsatsaronis et al. 2015):

Questions and answers were written by biomedical experts based on a controlled protocol and cross-validated against published PubMed abstracts. The organizing committee reported that the cross-consistency accuracy exceeded 0.85.

C. MedQA-USMLE:

Directly derived from the US Medical Licensing Examination, its question bank and answers are sourced from the official question bank, ensuring its inherent reliability without the need for additional reliability metrics.

D. Existing lexicon: UMLS

The Unified Medical Language System (UMLS), managed by the U.S. National Library of Medicine, contains over 3.5 million biomedical concepts and their synonyms, abbreviations, and canonical names. This project initializes a UMLS linker using the scispaCy en_core_sci_lg model. This linker maps detected entities to their corresponding UMLS Concept Unique Identifiers (CUIs)—for example, mapping "heart attack" to "C0027051: myocardial infarction" — and appends the canonical name to the original query. This lexicon satisfies the requirement of an "existing dictionary" and could serve as a foundational component for extending the methods implemented in the RAG process.

III. Model Architecture and Method Extension

A. TF-IDF statistical baseline:

We utilized the TfidfVectorizer (with max_features=50,000, ngram_range=(1,2), and sublinear_tf=True) from scikit-learn to retrieve documents by calculating cosine similarity, then the system extracts the highest-scoring sentence as the predicted answer. Since this baseline method does not involve a training phase, it is only used to quantify the performance improvement of neural network methods relative to traditional word matching techniques. A key anticipated limitation is that TF-IDF cannot identify semantically equivalent words—for example, the semantic equivalence between "heart attack" and "myocardial infarction"—which is precisely the core motivation for introducing the BERT and RAG architectures.

B. Supervised fine-tuning of PubMedBERT:

We adopted the microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract-fulltext model, which employs a 12-layer Transformer architecture with 768-dimensional hidden states, 12 attention heads, and approximately 110 million parameters. Unlike BioBERT, which is initialized based on a general-domain BERT, PubMedBERT's vocabulary and weights are trained from scratch on the entire PubMed database. This domain-specific pre-training ensures more comprehensive

coverage of medical terminology and more efficient sub-segmentation of specialized clinical terms.

1) Add two linear spans to the top of the category header:

```
H ∈ R^(seq_len × 768)        # BERT Output
start_logits = H · W_start   # W_start ∈ R^(768 × 1)
end_logits   = H · W_end     # W_end   ∈ R^(768 × 1)
```

2) The default training objective combines focal loss with label smoothing (implemented in medqa/training/custom_loss.py as FocalLabelSmoothingLoss), applied independently to the start and end logits and then averaged:

$L = 0.5 \cdot (\mathrm{FocalLS}(\mathrm{start_logits}, \mathrm{start_label}) + \mathrm{FocalLS}(\mathrm{end_logits}, \mathrm{end_label}))$, where $\mathrm{FocalLS}(x, y) = (1 - p_y)^{\gamma} \cdot [(1 - \varepsilon) \cdot \mathrm{CE}(x, y) + \varepsilon \cdot \mathrm{uniform_smooth}(x)]$. Default hyperparameters are focal_gamma $\gamma = 2.0$ and label_smoothing $\varepsilon = 0.1$ (set in config.py BERT_FINETUNE); setting $\gamma = \varepsilon = 0$ and switching loss_type to "ce" recovers plain cross-entropy.

The preprocessing flow performs whitespace normalization and control character cleanup on both the context and the answer. The context is then truncated to ensure that the combined length of the question and context does not exceed 512 tokens. If a character that exactly matches the answer is found in the text, the system records the start and end positions at the character level and maps them to token indices; otherwise, the start and end positions default to the [CLS] token (index 0).

This CLS backoff strategy follows the SQuAD v2.0 standard's guidelines for unanswerable samples, retaining approximately 40% of records that would otherwise be discarded, allowing BERT to derive valid gradients from these samples. During inference, a high probability value for the [CLS] token serves as a confidence signal, indicating a lack of a valid answer in the context. For particularly long documents, a sliding window method is used, with a maximum length of 512 tokens and a step size of 128, to ensure that each possible answer span appears at least once within the window.

C. LangChain's RAG pipeline：

TABLE II. TABLE B

Phase	*components*	*Function description*
Chunking	LangChain RecursiveCharacterTextSplitter	The PubMed digest is sliced into 512-character chunks, overlapping by 64 characters.
Query Expansion	scispaCy + UMLS linker	Map the patient's language to a UMLS canonical name and append it to the query (e.g., "heart attack" → +Myocardial Infarction).
Dense Retrieval	ChromaDB + BGE-M3	Query vectorization and return Top-10 similar chunks
Reranking	BGE-reranker-v2-m3	For each query and chunk, use a combined score and retain the top-3 most relevant results.
Generation	Qwen2.5-14B-Instruct (NF4)	4-bit quantized reasoning
		feeds the assembled context into an LLM to generate the answer.

In this project, Qwen 2.5-14B is used solely as an instruction-optimized answer generation model. Therefore, its primary use focuses on retrieval chains, prompt design, and evaluation protocols. While full fp16 inference with Qwen 2.5-14B requires approximately 28 GB of GPU memory, applying NF4 and double quantization using the BitsAndBytes library reduces this requirement to approximately 9 GB. This significantly lowers the hardware barrier to running the project, enabling the model to be deployed on a single consumer-grade GPU, such as an RTX 3090 or 4090. This optimization not only results in an accuracy loss of less than 2% but also allows the BGE reranker and dense retrieval embedding model to reside simultaneously on the same GPU, enabling an efficient end-to-end RAG process.

D. Method extension

1) Extension 1: Query Extension using UMLS Structured Ontologies

Common RAG systems typically perform intensive retrievals using raw queries. We extend this functionality by implementing the `expand_query_with_umls` function in `preprocessor.py`. This process includes: (1) detecting medical entities using scispaCy; (2) mapping these entities to UMLS CUIs via a linker; and (3) appending the canonical name to the query if it differs from the original text. For example, "treatment for heart attacks" is expanded to "... + myocardial infarction". This allows a single query to retrieve documents matching both patient-level language and specialized clinical terminology simultaneously without requiring additional training data.

2) Extension 2: Two-Stage Retrieval (Dual Encoder + Cross Encoder)

We employ a two-stage retrieval process. In the first stage, an encoder (BGE-M3) independently encodes the query and documents, simplifying the retrieval task to efficient vector dot product operations. This enables millisecond-level retrieval speeds in corpora containing millions of words. The second stage uses a cross encoder to re-rank the top-ranked candidate results to improve retrieval accuracy.

3) Extension 3: Answer type conditional hints

Different datasets require specific formats: PubMedQA requires single-word decisions, while BioASQ requires factual noun phrases. Without adaptation, PubMedQA's exact match (EM) score drops drastically to near zero. To address this, llm_qa.py dynamically switches between three system hint types based on `answer_type`, below are 4 cases:

a) `yesno`: Restricts output to {yes, no, maybe}, `max_new_tokens = 4`.

b) `factoid`: Extracts noun phrases of 1-8 words from the context, `max_new_tokens = 32`.

c) `Free`: Generates a concise summary (maximum 2 sentences) using the default lexical budget.

d) `answer_type` can be provided by `run_eval.py` or inferred via `_detect_answer_type()`. This targeted hint improves the EM of the RAG system on PubMedQA from 0 to 0.25; because PubMedQA gold answers are single tokens, this EM coincides with the F1 value reported for RAG on PubMedQA in Table D.

4) Key hyperparameters

TABLE III. TABLE C

Parameter	***value***	***Reason for selection***
learning_rate (BERT)	2e-5	BERT fine-tunes the standard interval; 1e-5 converges too slowly, and 5e-5 is prone to catastrophic forgetting on small datasets.
num_train_epochs	3	Approximately 16k joint samples are sufficient for convergence; more rounds tend to overfit to 1k PubMedQA.
batch_size	16	The maximum feasible value of 16 GB VRAM with fp16 and 512-token.
max_length / stride	512 / 128	BERT hard cap; 25% overlap for better span coverage.
chunk_size / overlap	512 / 64 char	Align BERT token budget; 12% overlap avoids sentence splitting across blocks.
retrieve_top_k	10	Provide enough candidates for reranker
rerank_top_k	3	≈1,500 characters, balancing LLM context window and signal dilution
max_new_tokens	4 / 32 / 512	Assignment based on answer_type (yesno / factoid / free)
temperature	0.0 (greedy)	Deterministic output ensures reproducible evaluation.

We also conducted 15 hyperparameter search trials using Optuna on a 1,000-sample training subset. The search space covered learning_rate, num_train_epochs, batch_size, warmup_ratio, weight_decay, focal_gamma, and label_smoothing, all layered on top of the focal + label-smoothing loss defined in Section III.B. The Optuna objective is to minimise the validation eval_loss (see scripts/hp_search.py). The best trial (trial 14, stored in reports/hp_search_results.json) reached eval_loss = 0.814 with learning_rate = 4.54×10^{-5}, num_train_epochs = 4, per_device_train_batch_size = 16, warmup_ratio = 0.032, weight_decay = 2.8×10^{-4}, focal_gamma = 2.61, and label_smoothing = 0.022.

Other detailed results and comparisons with default parameters can be found in Section 4.3.

IV. EXPERIMENT RESULT

During the evaluation phase, we used run_eval.py on a stratified test set containing 200 samples (random_state = 42, stratified by source across MedQA-USMLE, BioASQ, and PubMedQA). The 200 samples preserve the source proportions of the combined training pool (approximately USMLE 52%, BioASQ 44%, PubMedQA 4%), which leaves only 8 PubMedQA items in the subset; PubMedQA-specific numbers should therefore be read with wider statistical uncertainty than the other two. Each record was evaluated under four conditions: (1) TF-IDF baseline; (2) fine-tuned PubMedBERT for span extraction; (3) LLM + Gold Context diagnostic baseline; and (4) the full RAG pipeline. Reporting conditions (3) and (4) separately isolates retrieval errors from generation errors.

A. Quantitative assessment

The table below reports F1 scores for the four backends, split by dataset.

(Resource from: reports/evaluation_results_default.json)

TABLE IV. TABLE D

Model	***MedQA-USMLE (n=103)***	***BioASQ (n=89)***	***PubMedQA (n=8)***
TF-IDF baseline	0.008	0.040	0.000
Fine-tune PubMedBERT	0.136	0.697	0.188
LLM + Gold Context	0.226	0.626	0.375
Complete RAG	0.044	0.184	0.250

Complete RAG query index：Recall@1 = 0.155、

Recall@3 = Recall@5 = Recall@10 = 0.210, MRR = 0.178.

B. Interpretation of result:

1) Generate limitation:

LLM + Gold Context conditions (F1 = 0.410, BERTScore = 0.886) represents the inherent generation capability of Qwen-14B after eliminating retrieval errors. Its performance on MedQA-USMLE clinical reasoning questions outperforms extractive PubMedBERT (0.226 vs. 0.136), demonstrating the powerful reasoning ability of LLM in complex medical scenarios.

2) Extractive advantage:

The fine-tuned PubMedBERT model achieved the highest exact match (EM) score of 0.350. This is because PubMedBERT extracts span information directly from the context, which naturally aligns with the fact-based (phrase-based) answers in BioASQ (F1 = 0.697). However, this specialization comes at the cost of lower F1 and BERTScore values on free text tasks.

3) Retrieval Bottleneck:

The complete RAG pipeline achieves an end-to-end F1 score of only 0.115, approximately 0.30 points lower than the upper limit of LLM + Golden Context. Its Recall@1 is 0.155, meaning only 15.5% of queries successfully place true evidence first. Recall@3 and Recall@10 are both 0.21, indicating that the main bottleneck lies in the coarse-grained dual encoder retrieval stage. This represents a key area for future system optimization.

C. Qualitative evaluation

For each error prediction, a heuristic classification is performed, producing the following 5 common error categories:

TABLE V. TABLE E

Error type	***Definition***	***Example***	***Observed in***

WRONG_RETRIEVAL	Retrieved paragraphs do not contain answers.	Search for hypertension led to a paper on diabetes.	all three backends (< 10% each)
HALLUCINATION	Model confidently provides answers that are not supported by context.	RAG made up drug dosage not present in the context.	TF-IDF (52% of errors)
PARTIAL	Part corrects but incomplete or with misaligned boundaries	BERT shows ACE inhibitors but misses ARBs.	TF-IDF (17%), PubMedBERT (9%)
TERMINOLOGY_MISMATCH	Short prediction (≤ 5 tokens) with zero token-F1 against gold; typically a synonym or abbreviation mismatch.	BERT outputs "cholecalciferol"; gold is "1,25-dihydroxycholecalciferol".	PubMedBERT (84% of its errors)
OTHER	Does not fit any of the four categories above.	Residual failures (e.g. MCQ format predictions).	TF-IDF (25% of errors)

Qualitative observations and quantitative indicators corroborate each other:

Running medqa/evaluation/qualitative.py over the n = 200 predictions yields category counts saved to data/processed/baseline_errors.json and data/processed/bert_errors.json. TF-IDF produces 197 errors dominated by HALLUCINATION (103 cases, 52%) and OTHER (50 cases, 25%), because the baseline returns a whole-sentence MCQ option list that overlaps little with short gold answers; only 7 errors (3.5%) qualify as pure WRONG_RETRIEVAL. Fine-tuned PubMedBERT produces 121 errors overwhelmingly of type TERMINOLOGY_MISMATCH (102 cases, 84%): its extracted span is often lexically disjoint from the multi-word gold answer (for example, "cholecalciferol" vs. "1,25-dihydroxycholecalciferol"), giving token-F1 = 0 even when the answer is clinically close. The Full RAG pipeline was not re-categorised by qualitative.py, but its Recall@1 = 0.155 implies that retrieval-side errors are the dominant failure mode.

D. Hyperparameter search comparison

TABLE VI. TABLE F

Condition	*Default F1 (5b)*	*HP Best F1 (5b')*	*Δ*
Whole PubMedBERT F1	0.387	0.365	-0.022
MedQA-USMLE subset	0.136	0.070	-0.066
BioASQ subset	0.697	0.724	+0.027
PubMedQA subset	0.188	0.188	0.000

Optuna's best parameter raise in BioASQ, but MedQA-USMLE has significant decrease, and whole F1 score decrease 0.022. We judge it's "overfitting for a single aggregation target": data source mixing ratios for the validation slice and the test set are different. To avoid cherry-picking, our report will show two copies of results.

E. RAG component ablation

scripts/ablation.py file performs grid experiments on/off × Top-k ∈ {1,3,5} of UMLS query expansion (QE) and reranker on 60 identically distributed samples:

TABLE VII. TABLE G

Configuration	*EM*	*F1*	*BERTScore*	*Recall@1*	*Recall@3*	*MRR*
raw-query + dense@5	0.083	0.101	0.818	0.083	0.083	0.083
raw-query + rerank@5	0.083	0.103	0.821	0.117	0.167	0.137
query-exp + dense@5	0.033	0.051	0.814	0.083	0.083	0.083
query-exp + rerank@5	0.067	0.087	0.817	0.117	0.167	0.136

BGE cross-encoder reordering is worth retaining: Recall@1 improves by 41% (0.083 → 0.117), Recall@3 doubles (0.083 → 0.167), and MRR improves by 64% (0.083 → 0.137), at the cost of 3–4 times latency, which is an acceptable trade-off. UMLS query expansion yields negative results on the current test set: Under dense-only conditions, EM drops from 0.083 to 0.033, almost halved; even combined with the reranker, it is still inferior to the original query (0.067 vs 0.083). The presumed reason is that the expanded canonical name dilutes the BGE-M3 query embedding. We retain this feature but disable it by default; conditional triggering based on entity confidence can be explored in the future. Increasing rerank_top_k further only improves Recall without improving EM/F1: From Top-1 to Top-5, Recall@5 increases to 0.183, but EM/F1 remains basically unchanged, indicating that the LLM signal has been diluted by noise. The default rerank_top_k = 3 is a reasonable trade-off between delay and accuracy.

F. Command line testing

System allows both --question single question and --input document batch access, three backend can switch:

```
# Single question, with query resource output
uv run medqa --question "What is the first-line treatment for hypertension?" \
```

```
--mode rag --show-context

# batch question(one question per line)
uv run medqa --input questions.txt --mode rag --output answers.json

# support --mode {baseline, bert, rag}
```

G. Interactive demo

`main.py` is built on Gradio and can be interacted with at http://localhost:7860 after startup. Users can switch between TF-IDF, BERT, and RAG backends on the same issue. RAG mode also lists the top-3 reranked chunks as evidence.

```
uv run python main.py # Open in browser http://localhost:7860
```

The interface provides built-in example issues (such as "What are the symptoms of Type 2 diabetes?") and displays the model on/off using a radio selector. The accompanying reports/figures/ folder contains demo screenshots and evaluation visualizations (eval_comparison.png, rag_scores.png, confidence_calibration.png, etc.), which can be previewed directly in VS Code using Ctrl+KV.

V. Reasons for Method and Tool Selection

A. Choice of Domain-Specific Language Model: PubMedBERT

PubMedBERT was selected over general-purpose BERT and BioBERT due to its domain-specific pretraining. Unlike BioBERT, which continues pretraining from general BERT and retains a general English vocabulary, PubMedBERT is trained from scratch on large-scale biomedical corpora (PubMed abstracts and full texts).

As a result, medical terms such as hypercholesterolemia and corticosteroid are tokenized into fewer subwords, preserving their semantic integrity. Under the same fine-tuning budget, this leads to more effective representation learning and typically better performance on biomedical QA tasks.

B. Choice of Large Language Model: Qwen2.5-14B

Qwen2.5-14B was chosen as a balance between performance and computational efficiency. Compared to smaller models (e.g., 7B), it provides stronger reasoning ability, which is important for complex medical QA. Compared to larger models (e.g., 70B), it achieves comparable inference quality in many tasks while being significantly more efficient.

With NF4 quantization, Qwen2.5-14B can be deployed on a single GPU with around 10GB+ VRAM, making it feasible to run demonstrations on consumer-grade hardware. In contrast, 70B models require multiple GPUs even after quantization, which substantially increases the reproducibility barrier.

C. Vector Database Selection: ChromaDB

ChromaDB was selected instead of FAISS due to its ease of use and better support for reproducibility. ChromaDB automatically persists vector indexes to disk, eliminating the need for manual serialization and deserialization.

In addition, its HNSW-based retrieval achieves latency comparable to FAISS for the given dataset scale (~62k chunks), while providing a more streamlined development workflow.

D. Domain-Specific NLP Tools: scispaCy + UMLS

The project uses scispaCy with UMLS instead of general-purpose spaCy. The standard spaCy model (en_core_web_lg) is not trained on biomedical text and often fails to correctly identify medical entities.

In contrast, en_core_sci_lg is specifically trained on biomedical corpora and includes a built-in UMLS linker. This enables accurate recognition and normalization of medical entities without requiring external APIs or additional configuration.

E. Decoding Strategy: Greedy Decoding

Greedy decoding (i.e., do_sample = False) is used instead of sampling-based decoding to ensure deterministic outputs. This is critical for reproducible evaluation, as the same input always produces the same output.

Sampling-based methods introduce randomness, which leads to variability across runs and makes it difficult to fairly compare metrics such as Exact Match (EM) and F1.

F. Retrieval Strategy: Two-Stage Retrieval

A two-stage retrieval pipeline is adopted instead of using only a bi-encoder or a cross-encoder.

Bi-encoder: efficient but less accurate due to independent encoding of queries and documents

Cross-encoder: more accurate but computationally expensive and not scalable to large corpora

The two-stage approach first uses a bi-encoder to retrieve top-k candidates efficiently, and then applies a cross-encoder reranker for more precise ranking. This design balances efficiency and accuracy, and reflects common industry practice.

G. Training Strategy: No End-to-End RAG Training

End-to-end RAG training was not adopted due to data and resource limitations. Such training requires a differentiable retriever and a large-scale dataset consisting of questions, supporting documents, and answers.

In this project, the available dataset (~21k samples) is insufficient for stable joint training. Therefore, a modular design with a fixed retriever and a fixed LLM is used. This approach is more interpretable, easier to debug, and better aligned with reproducibility requirements.

VI. Reproduction steps

```
# 0. Install dependencies（Managed by pyproject.toml）
uv sync
# 1. Install the scispaCy UMLS model (first time only)
uv run python scripts/setup_scispacy.py
# 2. Building a RAG index（≈30 min，≈2 GB VRAM）
uv run python scripts/build_rag_index.py
# 3. Optuna HP Query（Optional，15 trials）
```

```
uv run python scripts/hp_search.py --trials 15 --train 1000 --resume
# 4. Fine-tune PubMedBERT
uv run python scripts/finetune_bert.py  # Default parameter
uv run python scripts/finetune_bert.py --use-best-params   # HP Optimal parameters
# 5. RAG ablation scan
uv run python scripts/ablation.py --n 60 --resume
# 6. End-to-end evaluation
uv run python run_eval.py --n 200 --tag default \
   --bert-checkpoint checkpoints/bert_qa --output-dir reports --resume
# 7. CLI Testing
uv run medqa --question "..." --mode rag
# 8. Launch the Gradio demo
uv run python main.py # Browser open http://localhost:7860
```

VII. Conclusion and Future Work

A. Key Findings

Fine-tuned PubMedBERT is the highest-ranking on EM (0.350), particularly on BioASQ factor-based question answering (F1 0.697).

The maximum generation (F1 0.410, BERTScore 0.886) is LLM + Gold Context; the complete RAG F1 is 0.115, which means that the bottleneck is in the retrieval layer (Recall@1 = 0.155).

The reordering of cross-encoders of BGE shows a stabilizing enhancement of the retrieval quality; UMLS query expansion does the opposite on the existing data source, and needs more specific triggering conditions.

Optuna HP search demonstrates a minor decrease in overall F1, with usual overfitting to a single aggregation goal.

B. Future Work

Retrieval side: Introduce a complete PubMed summary index; or introduce a hybrid retrieval of BM25 + dense. Query Expansion: UMLS expansion is conditioned on entity confidence and entity type; once expanded vectors are re-weighted to prevent subjecting the topic to dilution. Generation side: Citation highlighting is added to identify fragments of the response to particular chunks, enhancing traceability. Training Side: Joint retriever-reader joint training is explored (FiD/Atlas style). Assessment Side: Evaluations of the LLM-as-judge and of the medical expert are combined into one and the clinical usability is reported as a second dimension.

References

[1] Devlin, J, Chang, MW, Lee, K & Toutanova, K 2019, 'BERT: pre-training of deep bidirectional transformers for language understanding', Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 1, pp. 4171–4186, doi:10.18653/v1/N19-1423.

[2] Gu, Y, Tinn, R, Cheng, H, Lucas, M, Usuyama, N, Liu, X, Naumann, T, Gao, J & Poon, H 2021, 'Domain-specific language model pretraining for biomedical natural language processing', ACM Transactions on Computing for Healthcare, vol. 3, no. 1, pp. 1–23, doi:10.1145/3458754.

[3] Jin, Q, Dhingra, B, Liu, Z, Cohen, WW & Lu, X 2019, 'PubMedQA: a dataset for biomedical research question answering', Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, pp. 2567–2577, doi:10.18653/v1/D19-1259.

[4] Lewis, P, Perez, E, Piktus, A, Petroni, F, Karpukhin, V, Goyal, N, Küttler, H, Lewis, M, Yih, W, Rocktäschel, T, Riedel, S & Kiela, D 2020, 'Retrieval-augmented generation for knowledge-intensive NLP tasks', Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, doi:10.48550/arXiv.2005.11401.

[5] Nogueira, R & Cho, K 2019, 'Passage re-ranking with BERT', arXiv preprint arXiv:1901.04085, doi:10.48550/arXiv.1901.04085.

[6] Rajpurkar, P, Zhang, J, Lopyrev, K & Liang, P 2016, 'SQuAD: 100,000+ questions for machine comprehension of text', Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, Austin, Texas, pp. 2383–2392, doi:10.18653/v1/D16-1264.

[7] Rajpurkar, P, Jia, R & Liang, P 2018, 'Know what you don't know: unanswerable questions for SQuAD', Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, vol. 2, Melbourne, pp. 784–789, doi:10.18653/v1/P18-2124.

[8] Tsatsaronis, G, Balikas, G, Malakasiotis, P, Partalas, I, Zschunke, M, Alvers, MR, Weissenborn, D, Krithara, A, Petridis, S, Polychronopoulos, D, Almirantis, Y, Pavlopoulos, J, Baskiotis, N, Gallinari, P, Artières, T, Ngomo, AN, Heino, N, Gaussier, E, Barrio-Alvers, L, Schroeder, M, Androutsopoulos, I & Paliouras, G 2015, 'An overview of the BIOASQ large-scale biomedical semantic indexing and question answering competition', BMC Bioinformatics, vol. 16, no. 1, article 138, pp. 1–28, doi:10.1186/s12859-015-0564-6.

[9] Zhang, T, Kishore, V, Wu, F, Weinberger, KQ & Artzi, Y 2020, 'BERTScore: evaluating text generation with BERT', 8th International Conference on Learning Representations (ICLR 2020), Addis Ababa, doi:10.48550/arXiv.1904.09675.

[10] Dettmers, T, Pagnoni, A, Holtzman, A & Zettlemoyer, L 2023, 'QLoRA: efficient finetuning of quantized LLMs', Advances in Neural Information Processing Systems, vol. 36, pp. 10088–10115, doi:10.48550/arXiv.2305.14314.